

Activités d'Intégration de Compétences — Partie 6

Objectif général

Conclure le projet par sa mise en production sur Kubernetes en adoptant une **architecture Kubernetes-native** — c'est-à-dire en remplaçant les mécanismes Spring Cloud (Eureka, Feign, API Gateway) par les primitives natives de Kubernetes (Services, Ingress, DNS interne). Vous packagerez ensuite l'ensemble avec **Helm**, proposerez une architecture de déploiement sur **AWS Cloud**, et rédigerez un **README professionnel** qui documente la solution finale. Cette partie représente l'aboutissement du projet : une plateforme micro services complète, observable, packagée et prête à être déployée en production.

Q1 — Créer une branche Git `version-6` et ajouter le Sprint 6 dans Jira

Indication : Créez la branche `version-6` à partir de `version-5` et ouvrez un Sprint 6 dans votre projet Jira. Avant de commencer, prenez un moment pour planifier : cette partie touche à l'ensemble de l'architecture, il est donc important de définir clairement les User Stories, d'estimer leur complexité et d'organiser le sprint avec soin. Une bonne User Story pour cette partie pourrait être : *"En tant qu'ops, je veux déployer toute la plateforme sur Kubernetes avec une seule commande Helm, afin de garantir la reproductibilité des environnements."*

```
git checkout version-5
git checkout -b version-6
```

Q2 — Passer d'une architecture Spring Cloud à une architecture Kubernetes-native

Indication : C'est la question la plus conceptuellement importante de cette partie. Il faut d'abord comprendre *pourquoi* ce changement est pertinent, avant de savoir *comment* le réaliser.

Dans les parties précédentes, vous avez utilisé trois composants Spring Cloud : **Eureka** pour la découverte de services, **Feign** pour la communication HTTP entre services, et **Spring Cloud Gateway** comme point d'entrée. Ces outils résolvaient un vrai problème : dans un environnement où les adresses IP des services changent dynamiquement, il faut un mécanisme pour que les services se trouvent les uns les autres.

Mais Kubernetes résout exactement ce même problème, nativement et de façon plus robuste. Chaque `Service` Kubernetes possède un nom DNS stable (ex. `etudiant-service`) qui reste constant même si les pods sous-jacents sont recréés. Kubernetes fait lui-même le load balancing entre les pods via son composant `kube-proxy`. Et `Ingress` remplace l'API Gateway pour le routage externe. Conserver Eureka et Spring Cloud Gateway dans un cluster Kubernetes, c'est donc faire doublon : vous gérez deux systèmes de découverte de services en parallèle, ce qui ajoute de la complexité sans bénéfice.

Le passage à une architecture Kubernetes-native consiste donc à **retirer** ces couches Spring Cloud et à laisser Kubernetes prendre en charge leur rôle. Voici les actions concrètes à mener dans chaque micro service.

Dans chaque micro service Spring Boot, retirez les dépendances Spring Cloud du `pom.xml` :

```
<!-- Supprimez ces dépendances dans etudiant-service et grading-service -->
<!--
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
-->
```

Dans `application.yml`, remplacez les URL Feign dynamiques (qui passaient par Eureka) par des URL statiques qui utilisent le DNS interne Kubernetes. Dans Kubernetes, un service nommé `etudiant-service` dans le namespace `default` est résolu automatiquement par l'URL `http://etudiant-service:8081`. Vous n'avez plus besoin d'Eureka pour ça.

```
# grading-service/src/main/resources/application.yml
# Avant (avec Feign + Eureka) : l'URL était résolue dynamiquement par Eureka
# Après (Kubernetes-native) : on utilise directement le DNS interne K8s
clients:
  etudiant-service:
    url: http://etudiant-service:8081 # Résolu par le DNS interne Kubernetes
```

Remplacez vos FeignClients par un `RestClient` ou `WebClient` standard qui appelle directement l'URL configurée ci-dessus. L'interface `@FeignClient` disparaît au profit d'un simple client HTTP :

```
// grading-service : remplacement du FeignClient par RestClient
@Service
@RequiredArgsConstructor
public class EtudiantClient {

    // L'URL est injectée depuis application.yml – pas de magie Eureka.
    // Kubernetes se charge de résoudre "etudiant-service" vers le bon pod.
    @Value("${clients.etudiant-service.url}")
    private String etudiantServiceUrl;

    private final RestClient restClient = RestClient.create();

    public EtudiantDTO findById(Long id) {
        return restClient.get()
            .uri(etudiantServiceUrl + "/api/etudiants/{id}", id)
            .retrieve()
            .body(EtudiantDTO.class);
    }
}
```

Supprimez les projets `eureka-server` et `api-gateway` de votre dépôt — ils sont désormais remplacés respectivement par le DNS interne Kubernetes et par un `Ingress`. Mettez également à jour le `docker-compose.yml` pour retirer ces services (conservez-le pour le développement local, où vous pouvez utiliser les noms de services Docker comme substitut au DNS Kubernetes).

Q3 — Créer les manifests Kubernetes pour tous les micro services

Indication : Un manifest Kubernetes est un fichier YAML déclaratif qui décrit l'état désiré d'une ressource. Kubernetes lit ces fichiers et fait tout ce qu'il faut pour atteindre cet état. Il existe plusieurs types de ressources à connaître pour ce projet. Un **Deployment** décrit comment déployer vos conteneurs (quelle image, combien de replicas, quelles variables d'environnement). Un **Service** expose vos pods sur le réseau interne du cluster. Un **Ingress** est le point d'entrée externe qui route le trafic HTTP/HTTPS vers les bons Services. Un **Secret** stocke les données sensibles (mots de passe, tokens) encodées en base64. Un **PersistentVolumeClaim** (PVC) réclame de l'espace de stockage persistant pour les bases de données.

Organisez vos manifests dans un dossier `k8s/` à la racine du projet, avec un sous-dossier par composant pour garder l'ensemble lisible. Voici les manifests à produire pour chaque service.

Secrets (à créer en premier)

Commencez toujours par les Secrets, car les autres ressources en dépendent. Les valeurs sont encodées en base64 (`echo -n "mavaleur" | base64`).

```
# k8s/secrets/postgres-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: postgres-secret
type: Opaque
data:
  # Toutes les valeurs sont encodées en base64.
  # En production, utilisez un gestionnaire de secrets (AWS Secrets Manager,
  # HashiCorp Vault) plutôt que de committer ces valeurs dans Git.
  POSTGRES_DB: ZXR1ZG1hbnRzZGI=          # "etudiantsdb"
  POSTGRES_USER: cG9zdGdyZXM=          # "postgres"
  POSTGRES_PASSWORD: cGFzc3dvcmQ=      # "password"
---
# k8s/secrets/jwt-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: jwt-secret
type: Opaque
data:
  JWT_SECRET: bW9uX3NlY3JldF9qd3RfdHJlc19sb25n  # votre secret JWT en base64
```

PostgreSQL avec PersistentVolumeClaim

```

# k8s/postgres/postgres-pvc.yaml
# Le PVC réclame 1 Go de stockage persistant.
# Contrairement à la Partie 2 (sans persistance), ici les données survivent
# au redémarrage des pods – indispensable en production.
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
spec:
  accessModes:
    - ReadWriteOnce      # Un seul pod peut écrire à la fois (adapté à PostgreSQL)
  resources:
    requests:
      storage: 1Gi
---
# k8s/postgres/postgres-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
spec:
  replicas: 1             # PostgreSQL standalone – pas de réplication ici
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:15
          envFrom:
            - secretRef:
                name: postgres-secret  # Injecte les variables depuis le Secret
          volumeMounts:
            - name: postgres-storage
              mountPath: /var/lib/postgresql/data
      volumes:
        - name: postgres-storage
          persistentVolumeClaim:
            claimName: postgres-pvc
---
# k8s/postgres/postgres-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: postgres          # Ce nom est utilisé comme hostname par les autres services
spec:
  selector:
    app: postgres

```

```
ports:
  - port: 5432
    targetPort: 5432
type: ClusterIP      # Accessible uniquement depuis l'intérieur du cluster
```

Micro service étudiant

```

# k8s/etudiant-service/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: etudiant-service
spec:
  replicas: 2          # 2 replicas pour la haute disponibilité
  selector:
    matchLabels:
      app: etudiant-service
  template:
    metadata:
      labels:
        app: etudiant-service
    spec:
      containers:
        - name: etudiant-service
          image: <votre-username>/etudiant-service:2.0
          ports:
            - containerPort: 8081
          env:
            - name: SPRING_DATASOURCE_URL
              value: jdbc:postgresql://postgres:5432/etudiantsdb
            - name: SPRING_DATASOURCE_USERNAME
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: POSTGRES_USER
            - name: SPRING_DATASOURCE_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: POSTGRES_PASSWORD
            - name: SPRING_REDIS_HOST
              value: redis
      # Les probes permettent à Kubernetes de savoir si le pod est prêt
      # à recevoir du trafic (readiness) et s'il est encore vivant (liveness).
      readinessProbe:
        httpGet:
          path: /actuator/health/readiness
          port: 8081
          initialDelaySeconds: 30
          periodSeconds: 10
      livenessProbe:
        httpGet:
          path: /actuator/health/liveness
          port: 8081
          initialDelaySeconds: 60
          periodSeconds: 30
---
# k8s/etudiant-service/service.yaml
apiVersion: v1

```

```

kind: Service
metadata:
  name: etudiant-service
spec:
  selector:
    app: etudiant-service
  ports:
    - port: 8081
      targetPort: 8081
  type: ClusterIP

```

Créez des manifests similaires (Deployment + Service) pour les services suivants, en adaptant les noms, ports et variables d'environnement :

Service	Image	Port	Dépendances
grading-service	<username>/grading-service:2.0	8082	postgres
notification-service	<username>/notification-service:2.0	8083	kafka
auth-service	<username>/auth-service:2.0	3001	mongodb
frontend	<username>/frontend:2.0	3000	—
redis	redis:7	6379	—
mongodb	mongo:7	27017	—
kafka	confluentinc/cp-kafka:7.5.0	9092	zookeeper
zookeeper	confluentinc/cp-zookeeper:7.5.0	2181	—

PVC pour MongoDB

```

# k8s/mongodb/mongodb-pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongodb-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi

```

Ingress — point d'entrée unique

L'Ingress remplace l'ancienne Spring Cloud Gateway. Il route les requêtes HTTP entrantes vers les bons Services selon le chemin de l'URL. K3S inclut par défaut le contrôleur Ingress Traefik, vous n'avez donc rien d'autre à installer.

```
# k8s/ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: main-ingress
  annotations:
    # Ces annotations sont spécifiques à Traefik (inclus dans K3S).
    # Elles permettent de strip le préfixe de chemin avant de forwarder la requête.
    traefik.ingress.kubernetes.io/router.middlewares: default-strip-
prefix@kubernetescrd
spec:
  rules:
    - host: projet-etudiants.local    # Ajoutez cette entrée dans /etc/hosts sur votre
VM
      http:
        paths:
          - path: /api/etudiants
            pathType: Prefix
            backend:
              service:
                name: etudiant-service
                port:
                  number: 8081
          - path: /api/notes
            pathType: Prefix
            backend:
              service:
                name: grading-service
                port:
                  number: 8082
          - path: /auth
            pathType: Prefix
            backend:
              service:
                name: auth-service
                port:
                  number: 3001
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port:
                  number: 3000
```

Déploiement sur K3S

Une fois tous les manifests créés, déployez l'ensemble sur votre VM K3S dans l'ordre suivant pour respecter les dépendances :

```
# 1. Secrets en premier (les autres ressources en dépendent)
kubectl apply -f k8s/secrets/

# 2. Stockage (PVC) avant les pods qui les utilisent
kubectl apply -f k8s/postgres/postgres-pvc.yaml
kubectl apply -f k8s/mongodb/mongodb-pvc.yaml

# 3. Infrastructures de base (bases de données, brokers)
kubectl apply -f k8s/postgres/
kubectl apply -f k8s/mongodb/
kubectl apply -f k8s/redis/
kubectl apply -f k8s/zookeeper/
kubectl apply -f k8s/kafka/

# 4. Micro services applicatifs
kubectl apply -f k8s/etudiant-service/
kubectl apply -f k8s/grading-service/
kubectl apply -f k8s/notification-service/
kubectl apply -f k8s/auth-service/
kubectl apply -f k8s/frontend/

# 5. Ingress en dernier (les services qu'il route doivent exister)
kubectl apply -f k8s/ingress.yaml

# Vérification globale
kubectl get pods,services,ingress
```

Q4 — Packager les manifests Kubernetes avec Helm

Indication : Vous avez maintenant une vingtaine de fichiers YAML dans votre dossier `k8s/`. Si vous devez déployer la même application dans trois environnements différents (dev, staging, prod) avec des configurations légèrement différentes (nombre de replicas, taille de la base de données, noms de domaine), copier-coller ces fichiers en changeant les valeurs manuellement devient vite une source d'erreurs. **Helm** résout ce problème : c'est le gestionnaire de paquets pour Kubernetes. Il transforme vos manifests en **templates** paramétrables, et les regroupe dans un **Chart**. Vous définissez ensuite les valeurs spécifiques à chaque environnement dans un fichier `values.yaml`.

Installez Helm sur votre VM K3S :

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

Créez la structure du Chart avec la commande `helm create` , puis adaptez-la à votre projet :

```
helm create projet-etudiants
```

La structure générée que vous adapterez est la suivante :

```
helm/projet-etudiants/
├── Chart.yaml           # Métadonnées du Chart (nom, version, description)
├── values.yaml          # Valeurs par défaut paramétrables
├── templates/
│   ├── _helpers.tpl     # Fonctions Helm réutilisables (labels, noms)
│   ├── secrets.yaml
│   ├── postgres/
│   │   ├── deployment.yaml
│   │   ├── service.yaml
│   │   └── pvc.yaml
│   ├── etudiant-service/
│   │   ├── deployment.yaml
│   │   └── service.yaml
│   ├── ...              # Un sous-dossier par service
└── ingress.yaml
```

Le fichier `Chart.yaml` décrit votre Chart :

```
# helm/projet-etudiants/Chart.yaml
apiVersion: v2
name: projet-etudiants
description: Plateforme micro services de gestion des étudiants
type: application
version: 1.0.0          # Version du Chart (pas de l'application)
appVersion: "2.0.0"     # Version de votre application
```

Le fichier `values.yaml` centralise toutes les valeurs configurables. Chaque template Helm y fera référence via la syntaxe `{{ .Values.xxx }}` , ce qui permet de changer une configuration sans toucher aux templates :

```

# helm/projet-etudiants/values.yaml
replicaCount:
  etudiantService: 2
  gradingService: 1
  frontend: 1

images:
  etudiantService:
    repository: <votre-username>/etudiant-service
    tag: "2.0"
    pullPolicy: IfNotPresent
  gradingService:
    repository: <votre-username>/grading-service
    tag: "2.0"
  frontend:
    repository: <votre-username>/frontend
    tag: "2.0"
  authService:
    repository: <votre-username>/auth-service
    tag: "2.0"

postgres:
  storage: 1Gi
  database: etudiantsdb

ingress:
  host: projet-etudiants.local

# Fichier values-prod.yaml (surcharge pour la production)
# replicaCount:
#   etudiantService: 3
#   gradingService: 2

```

Voici un exemple de template Helm pour le Deployment d' `etudiant-service` . Remarquez comment `{{ .Values.xxx }}` est utilisé pour injecter les valeurs, et comment `{{ include "projet-etudiants.labels" . }}` réutilise des labels communs définis dans `_helpers.tpl` :

```
# helm/projet-etudiants/templates/etudiant-service/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: etudiant-service
  labels:
    {{- include "projet-etudiants.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount.etudiantService }}
  selector:
    matchLabels:
      app: etudiant-service
  template:
    metadata:
      labels:
        app: etudiant-service
    spec:
      containers:
        - name: etudiant-service
          image: "{{ .Values.images.etudiantService.repository }}:{{
.Values.images.etudiantService.tag }}"
          imagePullPolicy: {{ .Values.images.etudiantService.pullPolicy }}
          ports:
            - containerPort: 8081
```

Pour déployer avec Helm, validez d'abord le rendu des templates avant d'appliquer :

```
# Valider les templates sans déployer (dry run)
helm template projet-etudiants ./helm/projet-etudiants/

# Déployer pour la première fois
helm install projet-etudiants ./helm/projet-etudiants/

# Mettre à jour après modification des valeurs ou des templates
helm upgrade projet-etudiants ./helm/projet-etudiants/

# Déployer avec des valeurs de surcharge (ex. production)
helm upgrade projet-etudiants ./helm/projet-etudiants/ \
  --values ./helm/projet-etudiants/values-prod.yaml

# Consulter l'état du déploiement
helm status projet-etudiants

# Désinstaller proprement (supprime toutes les ressources K8s créées par le Chart)
helm uninstall projet-etudiants
```

Q5 — Proposer une architecture AWS Cloud pour le déploiement

Indication : Transposer une architecture Kubernetes locale (K3S sur VM) vers AWS ne consiste pas à "juste mettre les mêmes fichiers sur le cloud". Il s'agit de choisir les services AWS managés qui remplacent avantageusement les composants que vous gérez vous-même, de manière à réduire la charge opérationnelle et à bénéficier de la haute disponibilité et de l'élasticité du cloud.

Voici l'architecture AWS recommandée pour votre plateforme, service par service.

Amazon EKS (Elastic Kubernetes Service) remplace votre cluster K3S local. EKS est un Kubernetes managé : AWS gère le control plane (API Server, etcd, scheduler) et vous ne gérez que vos worker nodes. Vous pouvez réutiliser vos manifests Kubernetes et votre Chart Helm sans modification : EKS est du Kubernetes standard.

Amazon RDS for PostgreSQL remplace votre pod PostgreSQL dans Kubernetes. RDS offre les sauvegardes automatiques, le failover multi-AZ, les mises à jour de sécurité automatiques, et le monitoring intégré — tout ce qu'il faudrait configurer manuellement avec un pod PostgreSQL stateful dans K8s. Votre `etudiant-service` pointe simplement vers l'endpoint RDS à la place de l'adresse du Service Kubernetes.

Amazon ElastiCache for Redis remplace votre pod Redis dans le cluster. Comme RDS, il offre la haute disponibilité, le failover automatique et le monitoring sans configuration manuelle.

Amazon DocumentDB (compatible MongoDB) remplace votre pod MongoDB pour l'`auth-service`. L'API est compatible avec MongoDB, donc aucun changement de code n'est nécessaire dans votre service Node.js.

Amazon MSK (Managed Streaming for Apache Kafka) remplace vos pods Kafka et Zookeeper. MSK gère la réplication, les brokers, et Zookeeper de manière entièrement transparente.

AWS Application Load Balancer (ALB) + AWS Load Balancer Controller remplace votre Ingress Traefik. Le ALB Ingress Controller pour EKS crée automatiquement un ALB AWS quand vous déployez une ressource Ingress dans EKS — il s'intègre nativement avec ACM pour le TLS et avec WAF pour la sécurité.

Amazon ECR (Elastic Container Registry) remplace Docker Hub pour stocker vos images Docker. ECR est privé par défaut et s'intègre naturellement avec EKS via IAM.

AWS Secrets Manager remplace vos Kubernetes Secrets pour les données sensibles (mots de passe, clés JWT). Avec le External Secrets Operator, vos pods Kubernetes récupèrent automatiquement les secrets depuis AWS Secrets Manager sans que vous ayez à stocker de valeurs sensibles dans des fichiers YAML.

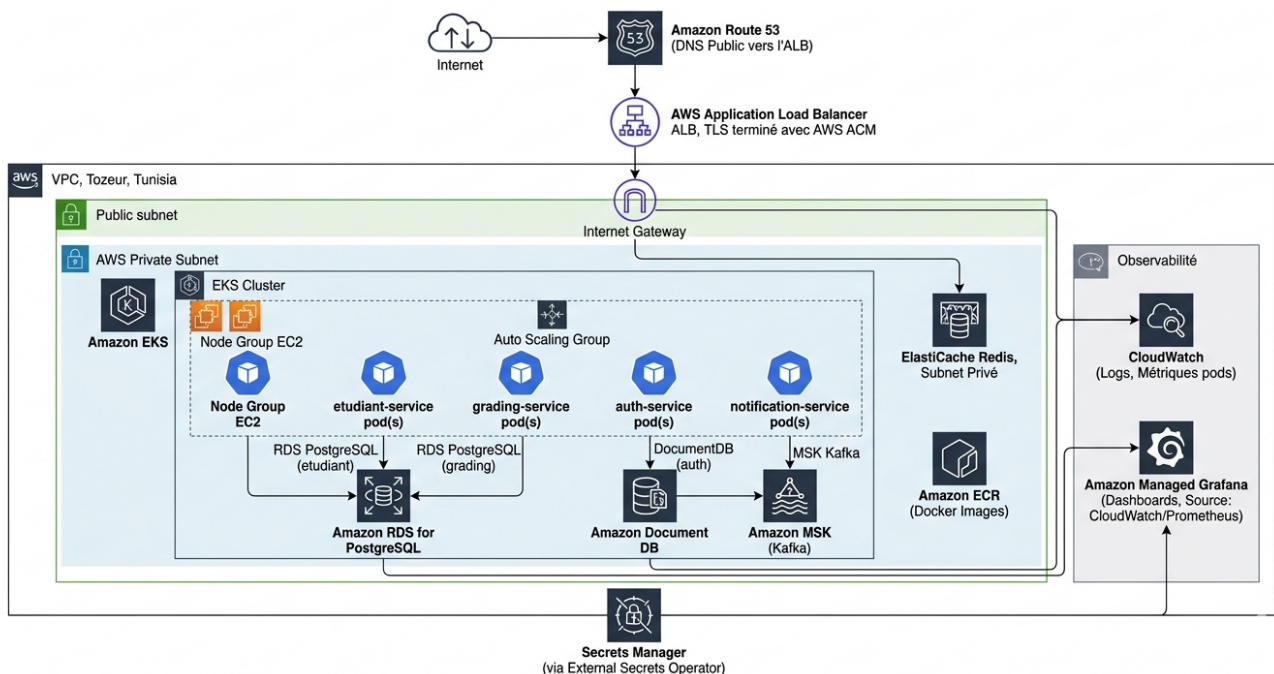
Le schéma d'architecture global est le suivant :

```

Internet
|
▼
[Route 53] → DNS public vers l'ALB
|
▼
[Application Load Balancer] ← TLS terminé ici avec AWS ACM
|
▼
[EKS Cluster – VPC privé]
├── Node Group (EC2 Auto Scaling)
│   ├── frontend pod(s)
│   ├── etudiant-service pod(s) → [RDS PostgreSQL – subnet privé]
│   ├── grading-service pod(s) → [RDS PostgreSQL – subnet privé]
│   ├── auth-service pod(s) → [DocumentDB – subnet privé]
│   └── notification-service → [MSK Kafka – subnet privé]
└── Observabilité
    ├── [Amazon CloudWatch] ← Logs + métriques des pods
    └── [Amazon Managed Grafana] ← Tableaux de bord (source : CloudWatch /
Prometheus)

[ElastiCache Redis] ← dans le même VPC, subnet privé
[ECR] ← images Docker
[Secrets Manager] ← secrets injectés via External Secrets Operator
  
```

AWS Microservices Architecture with EKS



Pour le déploiement sur EKS, adaptez votre workflow de déploiement Helm :

```
# Authentification et configuration du cluster EKS
aws eks update-kubeconfig --name projet-etudiants-cluster --region eu-west-1

# Déploiement avec Helm en ciblant les images ECR et les endpoints AWS
helm upgrade --install projet-etudiants ./helm/projet-etudiants/ \
  --values ./helm/projet-etudiants/values-prod.yaml \
  --set images.etudiantService.repository=<account-id>.dkr.ecr.eu-west-1.amazonaws.com/etudiant-service \
  --set postgres.host=<rds-endpoint>.rds.amazonaws.com
```

Q6 — Rédiger un README professionnel

Indication : Le README est souvent la première chose qu'un recruteur, un collègue ou un contributeur lit de votre projet. Un README professionnel ne se contente pas de lister des commandes : il raconte l'histoire du projet, explique les choix architecturaux, et guide le lecteur du contexte général jusqu'à l'exécution en passant par l'architecture. Voici la structure recommandée, avec ce que chaque section doit contenir.

Projet Étudiants – Plateforme Micro Services

Description

Présentation en 3-4 phrases du projet, de son contexte pédagogique et des technologies principales utilisées.

Architecture

[Insérez ici un schéma d'architecture global – capture d'écran ou diagramme exporté depuis draw.io / Excalidraw]

Description textuelle des micro services et de leurs interactions.

Stack technique

Composant	Technologie
---	---
Micro service étudiant	Spring Boot 4, JDK 25, PostgreSQL, Redis
Micro service notes	Spring Boot 4, PostgreSQL
Micro service notifications	Spring Boot 4, Apache Kafka
Micro service auth	Node.js, Express, MongoDB, JWT
Frontend	Next.js, Tailwind CSS
Application mobile	Flutter / React Native
Orchestration	Kubernetes (K3S / EKS), Helm
Observabilité	ELK Stack, Prometheus, Grafana
CI/CD	GitHub Actions, Xray, Jira

Lancement rapide

Prérequis

- Docker Desktop ≥ 24.0
- Java 25, Maven 3.9+
- Node.js 20+
- kubectl, Helm 3

Avec Docker Compose (développement local)

```
git clone https://github.com/<username>/projet-etudiants.git
cd projet-etudiants
docker compose up --build
```

Accès aux services :

- API principale : http://localhost:8080
- Frontend : http://localhost:3000
- Kibana (logs) : http://localhost:5601
- Grafana (métriques) : http://localhost:3001
- Eureka Dashboard : retiré (architecture K8s-native)

Avec Helm sur K3S

```
helm install projet-etudiants ./helm/projet-etudiants/
kubectl get pods --watch
```

Captures d'écran

[Insérez ici des captures d'écran de : le frontend Next.js, l'application mobile, Swagger UI, Kibana, Grafana, le board Jira]

Tests

Tests unitaires + intégration + couverture JaCoCo

```
cd api-spring-boot && mvn verify
```

Tests E2E Cypress (stack Docker démarrée au préalable)

```
cd frontend && npx cypress run
```

Tests de stress Gatling

```
mvn gatling:test
```

Structure du dépôt

[Arbre de la structure du projet]

Auteur

Nom, formation, année

Structure finale du dépôt GitHub

```

/projet-etudiants/
├── api-spring-boot/           # Micro service étudiant (Spring Boot)
├── grading-service/          # Micro service notes (Spring Boot)
├── notification-service/     # Micro service notifications + Kafka consumer
├── auth-service/             # Micro service auth (Node.js + Express + MongoDB)
├── frontend/                 # Application Next.js
│   └── cypress/e2e/          # Tests E2E Cypress
├── mobile-app/               # Application Flutter ou React Native
├── helm/
│   └── projet-etudiants/    # Chart Helm packagisant toute la plateforme
│       ├── Chart.yaml
│       ├── values.yaml
│       ├── values-prod.yaml
│       └── templates/
├── k8s/                      # Manifests Kubernetes bruts (avant Helm)
│   ├── secrets/
│   ├── postgres/
│   ├── mongodb/
│   ├── redis/
│   ├── kafka/
│   ├── etudiant-service/
│   ├── grading-service/
│   ├── notification-service/
│   ├── auth-service/
│   ├── frontend/
│   └── ingress.yaml
├── observability/
│   ├── logstash/pipeline/
│   └── prometheus/
├── .github/
│   ├── workflows/
│   │   └── test-and-report.yml
│   ├── ISSUE_TEMPLATE/
│   └── pull_request_template.md
├── docker-compose.yml
└── README.md                 # README professionnel avec captures d'écran

```

💡 **Réflexion finale** : en arrivant à cette partie, vous avez parcouru l'intégralité du cycle de vie d'une application moderne — du développement local à la mise en production cloud, en passant par les tests, la conteneurisation, l'orchestration et l'observabilité. L'architecture Kubernetes-native que vous avez adoptée en Q2 illustre un principe important : les bonnes abstractions évoluent avec le contexte. Spring Cloud était la bonne solution quand Kubernetes n'était pas encore omniprésent. Aujourd'hui, déléguer la découverte de services et le load balancing à Kubernetes simplifie votre code applicatif et réduit la surface de configuration à maintenir. C'est ce type de raisonnement architectural — comprendre *pourquoi* un outil existe et *quand* le remplacer — qui distingue un ingénieur DevOps expérimenté d'un simple exécutant.